

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 03 -

DESIGN PATTERNS EM OO

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	22
REFERÊNCIAS.....	23

EMSE

O QUE VEM POR AÍ?

Nesta aula veremos o que são padrões de projeto e para que eles servem. Iremos explorar suas categorias e implementar algumas delas para que possamos ver sua aplicabilidade.



HANDS ON

Iniciaremos explicando o que são padrões de projetos e suas categorias. Estudaremos as principais e faremos uma implementação utilizando alguns desses padrões.



SAIBA MAIS

O QUE SÃO PADRÕES DE PROJETO

Padrões de projetos são soluções para problemas comuns em modelagem de sistemas. Eles podem ser aplicados em diversas situações no desenvolvimento de software, ajudando a resolver problemas recorrentes com eficiência e reuso.

Cada padrão de projeto tem uma solução testada e comprovada, que pode ser adaptada a necessidades específicas de um projeto. Utilizando padrões de projeto, é esperado que se tenha um design de software mais limpo, legível e fácil de manter.

Categorias

Os padrões de projeto podem ser classificados em três categorias principais:

- **Padrões Criacionais:** são os padrões que tratam da criação de objetos. Seu objetivo é encapsular a complexidade da criação de objetos e fornecer flexibilidade para reutilização de código. Exemplos: Singleton e Factory Method.
- **Padrões Estruturais:** têm o objetivo de compor classes e objetos para formar estruturas maiores. Eles ajudam a garantir que, ao combinar diferentes partes do software, o sistema permaneça flexível e eficiente. Exemplos: Adapter e Decorator.
- **Padrões Comportamentais:** tratam da comunicação entre objetos e a responsabilidade entre eles. Com esses padrões, é possível definir como os objetos interagem e como as responsabilidades são distribuídas. Exemplos: Chain of Responsibility e Strategy.

Criacionais - Singleton

Esse padrão garante que uma classe tenha somente uma instância no sistema e fornece um ponto de acesso global a ela. Isso é útil quando precisamos acessar o mesmo objeto dentro do sistema, independente do contexto ou da camada.

Suponha que temos uma classe de nome “MediaNotaSystem” responsável por gerir notas de alunos, como no Código 1.

```
public class MediaNotaSystem {  
    // Instância única da classe GradeSystem  
    private static MediaNotaSystem instance;  
  
    // Construtor privado para evitar instanciação externa  
    private MediaNotaSystem() {  
        // Inicialização dos dados do sistema de notas  
    }  
  
    // Método público estático que retorna a única instância da classe  
    public static synchronized MediaNotaSystem getInstance() {  
        if (instance == null) {  
            instance = new MediaNotaSystem();  
        }  
        return instance;  
    }  
  
    // Métodos para gerenciar as notas dos alunos  
    public void addGrade(String studentId, double grade) {  
        // Lógica para adicionar uma nota para um aluno  
    }  
  
    public double getAverageGrade(String studentId) {  
        // Lógica para calcular a média das notas de um aluno  
        return 0.0;  
    }  
    // Outros métodos necessários para o sistema de notas  
}
```

Código 1 — Classe de nome “MediaNotaSystem”
Fonte: elaborado pelo autor (2024)

Temos o construtor que inicializa o objeto. Em seguida, temos o método de acesso global que obtém o objeto. Perceba que ele sempre retorna o mesmo objeto. Caso não exista, ele cria. Por ser estático, pode ser chamado sem ter uma instância

da classe. Os demais métodos servem para executar a lógica da nota e não foram implementados neste exemplo.

O código 2 demonstra o uso desta classe.

```
public class Main {  
  
    public static void main(String[] args) {  
        // Obtém a única instância do GradeSystem  
        MediaNotaSystem mediaNotaSystem = MediaNotaSystem.getInstance();  
  
        // Adiciona notas para alunos  
        mediaNotaSystem.addGrade("123", 95.0);  
        mediaNotaSystem.addGrade("123", 85.0);  
  
        // Calcula a média das notas de um aluno  
        double average = mediaNotaSystem.getAverageGrade("123");  
        System.out.println("Média do aluno 123: " + average);  
    }  
}
```

Código 2 — Uso da classe
Fonte: elaborado pelo autor (2024)

Neste exemplo estamos garantindo que apenas uma instância de “MediaNotaSystem” seja usada em todo o programa. Essa é a essência do Singleton.

Criacionais – Factory Method

Esse padrão ajuda a desacoplar a criação dos objetos do seu uso, facilitando a manutenção e a extensão do sistema. Vamos para um exemplo em um sistema de notas.

Primeiro criamos uma interface ou classe abstrata que representa uma nota. Veja o código 3.

```
public abstract class Nota {  
    protected double value;  
  
    public Nota(double value) {
```

```
this.value = value;
}

public double getValue() {
    return value;
}

public abstract String getType();
}
```

Código 3 — Interface ou classe abstrata
Fonte: elaborado pelo autor (2024)

Agora criamos as classes concretas das notas. O Código 4 ilustra duas classes.

```
public class NotaProva extends Nota {

    public NotaProva(double value) {
        super(value);
    }

    @Override
    public String getType() {
        return "Prova";
    }
}
```



```
public class NotaTrabalho extends Nota {  
    public NotaTrabalho(double value) {  
        super(value);  
    }  
  
    @Override  
    public String getType() {  
        return "Trabalho";  
    }  
}
```

Código 4 — Classes
Fonte: elaborado pelo autor (2024)

Em seguida criamos a classe abstrata da fábrica de nota. Vide Código 5.

```
public abstract class NotaFactory {  
    public abstract Nota criarNota(double value);  
}
```

Código 5 — Classe abstrata da fábrica de nota
Fonte: elaborado pelo autor (2024)

Agora criamos as classes concretas das fábricas de notas. O Código 6 demonstra essas classes.

```
public class NotaProvaFactory extends NotaFactory {  
    @Override  
    public Nota criarNota(double value) {  
        return new NotaProva(value);  
    }  
}  
  
public class NotaTrabalhoFactory extends NotaFactory {  
    @Override  
    public Nota criarNota(double value) {  
        return new NotaTrabalho(value);  
    }  
}
```

Código 6 — Classes concretas das fábricas de notas

Fonte: Elaborado pelo Autor (2024)

Agora podemos usar as fábricas para que criem nossos objetos. O código 7 demonstra o uso delas.

```
public class Main {  
  
    public static void main(String[] args) {  
        // Fábrica para criar notas de exame  
        NotaFactory notaProvaFactory = new NotaProvaFactory();  
        Nota notaProva = notaProvaFactory.criarNota(90.0);  
        System.out.println("Tipo de nota: " + notaProva.getType() + ", Valor: " + notaProva.getValue());  
  
        // Fábrica para criar notas de tarefa  
        NotaFactory trabalhoFactory = new NotaTrabalhoFactory();  
        Nota notaTrabalho = trabalhoFactory.criarNota(85.0);  
        System.out.println("Tipo de nota: " + notaTrabalho.getType() + ", Valor: " +  
        notaTrabalho.getValue());  
    }  
}
```

Código 7 — Uso de fábricas para criar objetos
Fonte: Elaborado pelo Autor (2024)

Estrutural – Adapter

Esse padrão permite que duas classes incompatíveis entre si trabalhem juntas. O padrão atua como um adaptador que converte a interface de uma classe em outra.

Vamos considerar um exemplo em que precisamos integrar nosso sistema a uma biblioteca (“lib”) externa. Essa biblioteca tem uma interface diferente do que precisamos, mas possui implementações relevantes ao projeto. Porém, pode ser que em breve usemos outra biblioteca e não queremos ter que mudar o “core” do projeto.

Primeiramente criamos a interface que será usada pelo “core” do sistema (código 8).

```
public interface Nota {  
    double getValue();  
    String getTipoNota();  
}
```

```
}
```

Código 8 — Interface que será usada pelo “core” do sistema
Fonte: Elaborado pelo Autor (2024)

Agora temos a biblioteca externa. Considere que ela não foi desenvolvida necessariamente pelo nosso time de desenvolvedores e que talvez o detalhe da implementação nem fosse relevante, pois queremos o resultado dela. Ela está ilustrada no código 9.

```
public class EmissorNotasExternos {  
    private int score;  
  
    public EmissorNotasExternos(int score) {  
        this.score = score;  
    }  
  
    public int getNota() {  
        return score;  
    }  
  
    public String getNotaDescricao() {  
        if (score >= 90) return "A";  
        else if (score >= 80) return "B";  
        else if (score >= 70) return "C";  
        else if (score >= 60) return "D";  
        else return "F";  
    }  
}
```

Código 9 — Biblioteca externa
Fonte: elaborado pelo autor (2024)

Agora criamos o adaptador que deve utilizar a biblioteca externa (código 10). Note que ele implementa a interface “Nota”. Assim, usando a interface, teremos o acesso à biblioteca externa encapsulado para nosso sistema.

```
public class NotaAdapter implements Nota {  
    private EmissorNotasExternos externalGrade;
```

```
public NotaAdapter(EmissorNotasExternos externalGrade) {  
    this.externalGrade = externalGrade;  
}  
  
@Override  
public double getValue() {  
    return externalGrade.getNota();  
}  
  
@Override  
public String getTipoNota() {  
    return externalGrade.getNotaDescricao();  
}  
}
```

Código 10 — Adaptador que deve utilizar a biblioteca externa
Fonte: elaborado pelo autor (2024)

Agora podemos usufruir da biblioteca externa sem acoplar o core do projeto a ela. Veja no código 11 como ficou o programa.

```
public class Main {  
  
    public static void main(String[] args) {  
        // Instanciando a classe da biblioteca externa  
        EmissorNotasExternos externalGrade = new EmissorNotasExternos(85);  
  
        // Usando o Adapter para integrar com o nosso sistema  
        Nota nota = new NotaAdapter(externalGrade);  
  
        // Utilizando os métodos da interface Nota  
        System.out.println("Nota: " + nota.getValue());  
        System.out.println("Descrição da Nota: " + nota.getTipoNota());  
    }  
}
```

Código 11 — Programa
Fonte: elaborado pelo autor (2024)

O mais interessante desse padrão é a possibilidade de mudarmos a implementação do adaptador sem precisar mudar o core do projeto. Por exemplo, poderíamos criar outro adaptador que usasse outra biblioteca e nosso projeto continuaria funcionando normalmente.

Estrutural – Decorator

Este padrão permite adicionar comportamento em um objeto sem afetar o comportamento de outros objetos da mesma classe. Ele é muito usado para aderir ao princípio aberto/fechado (Open Closed), que afirma que as classes devem estar abertas para extensão, mas fechadas para modificação. Ou seja, adicionar novos comportamentos sem alterar (ou alterando pouco) o código já existente.

No nosso exemplo, adicionaremos funcionalidades extras a notas, como calcular nota com peso e notas com bônus. Primeiramente vamos definir nossa interface de Nota. O código 12 ilustra essa interface.

```
public interface Nota {  
    double getValue();  
    String getDescription();  
}
```

Código 12 — Interface de Nota
Fonte: elaborado pelo autor (2024)

Implementamos a classe concreta de uma nota (código 13):

```
public class NotaBasica implements Nota {  
  
    private double value;  
  
    public NotaBasica(double value) {  
        this.value = value;  
    }  
  
    @Override  
    public double getValue() {  
        return value;  
    }  
}
```

```
@Override  
public String getDescription() {  
    return "Nota básica";  
}  
}
```

Código 13 — Classe concreta de uma nota
Fonte: elaborado pelo autor (2024)

Implementamos a classe abstrata da classe decorator na Nota (Código 14).

```
public abstract class NotaDecorator implements Nota {  
  
    protected Nota decoratedNota;  
  
    public NotaDecorator(Nota decoratedNota) {  
        this.decoratedNota = decoratedNota;  
    }  
  
    @Override  
    public double getValue() {  
        return decoratedNota.getValue();  
    }  
  
    @Override  
    public String getDescription() {  
        return decoratedNota.getDescription();  
    }  
}
```

Código 14 — Classe abstrata
Fonte: Elaborado pelo Autor (2024)

Construímos então as classes decorator concretas (código 15 e 16). Notem que elas possuem implementações que estendem o comportamento da Nota Padrão.

```
public class NotaBonusDecorator extends NotaDecorator {  
    private double bonus;  
  
    public NotaBonusDecorator(Nota decoratedNota, double bonus) {
```

```
    super(decoratedNota);  
    this.bonus = bonus;  
}  
  
@Override  
public double getValue() {  
    return decoratedNota.getValue() + bonus;  
}  
  
@Override  
public String getDescription() {  
    return decoratedNota.getDescription() + " com bônus";  
}  
}
```

Código 15 — Classes decorator concreta (1)
Fonte: elaborado pelo autor (2024)

```
public class NotaComPesoDecorator extends NotaDecorator {  
    private double peso;  
  
    public NotaComPesoDecorator(Nota decoratedNota, double peso) {  
        super(decoratedNota);  
        this.peso = peso;  
    }  
  
    @Override  
    public double getValue() {  
        return decoratedNota.getValue() * peso;  
    }  
  
    @Override  
    public String getDescription() {  
        return decoratedNota.getDescription() + " com peso";  
    }  
}
```

Código 16 — Classes decorator concreta (2)
Fonte: elaborado pelo autor (2024)

Por fim, o código 17 demonstra o uso do padrão decorator:

```
public class Main {  
    public static void main(String[] args) {  
        // Cria uma nota básica  
        Nota basicGrade = new NotaBasica(85);  
  
        // Adiciona um bônus à nota  
        Nota bonusGrade = new NotaBonusDecorator(basicGrade, 5);  
        System.out.println("Valor: " + bonusGrade.getValue());  
        System.out.println("Descrição: " + bonusGrade.getDescription());  
  
        // Adiciona um peso à nota com bônus  
        Nota weightedGrade = new NotaComPesoDecorator(bonusGrade, 1.2);  
        System.out.println("Valor: " + weightedGrade.getValue());  
        System.out.println("Descrição: " + weightedGrade.getDescription());  
    }  
}
```

Código 17 — Uso do padrão Decorator
Fonte: elaborado pelo autor (2024)

Comportamental – Chain Of Responsibility

Esse padrão permite que múltiplos objetos processem uma solicitação sem que o remetente desta solicitação saiba qual objeto vai manipulá-la. Os objetos são encadeados e a solicitação passa por essa cadeia até que um dos objetos a processe.

Vamos ao exemplo. O primeiro passo é definir a classe base (Handler).

```
public abstract class AvaliadorHandler {  
    protected AvaliadorHandler nextHandler;  
  
    public void setNextHandler(AvaliadorHandler nextHandler) {  
        this.nextHandler = nextHandler;  
    }  
  
    public abstract void handleRequest(double grade);  
}
```

Código 18 — Definir a classe base
Fonte: elaborado pelo autor (2024)

Agora criaremos os handlers concretos. Note que cada um deles executa um processamento dada uma condição e, dependendo, encerra o fluxo ou chama o próximo handler, como vemos nos códigos 19, 20 e 21.

```
public class AprovadorAutomaticoHandler extends AvaliadorHandler {

    @Override
    public void handleRequest(double grade) {
        if (grade >= 90) {
            System.out.println("Nota " + grade + " aprovada automaticamente.");
        } else if (nextHandler != null) {
            nextHandler.handleRequest(grade);
        }
    }
}
```

Código 19 — Handlers concretos (1)
Fonte: elaborado pelo autor (2024)

```
public class RevisorHandler extends AvaliadorHandler {

    @Override
    public void handleRequest(double grade) {
        if (grade >= 70 && grade < 90) {
            System.out.println("Nota " + grade + " precisa de revisão.");
        } else if (nextHandler != null) {
            nextHandler.handleRequest(grade);
        }
    }
}
```

Código 20 — Handlers concretos (2)
Fonte: Elaborado pelo Autor (2024)

```
public class RejeicaoHandler extends AvaliadorHandler {

    @Override
    public void handleRequest(double grade) {
        if (grade < 70) {
            System.out.println("Nota " + grade + " reprovada.");
        }
    }
}
```

```
    } else if (nextHandler != null) {  
        nextHandler.handleRequest(grade);  
    }  
}  
}
```

Código 21 — Handlers concretos (3)
Fonte: elaborado pelo autor (2024)

Agora iremos utilizar nossos handlers para efetuar a validação de notas. O código 22 demonstra isso.

```
public class Main {  
    public static void main(String[] args) {  
        // Criar os handlers  
        AvaliadorHandler aprovadorAutomaticoHandler =  
            new AprovadorAutomaticoHandler();  
        AvaliadorHandler revisorHandler = new RevisorHandler();  
        AvaliadorHandler rejeicaoHandler = new RejeicaoHandler();  
  
        // Configurar a cadeia de responsabilidade  
        aprovadorAutomaticoHandler.setNextHandler(revisorHandler);  
        revisorHandler.setNextHandler(rejeicaoHandler);  
  
        // Processar diferentes notas  
        double[] grades = {95, 85, 65};  
  
        for (double grade : grades) {  
            aprovadorAutomaticoHandler.handleRequest(grade);  
        }  
    }  
}
```

Código 22 — Validação de notas
Fonte: elaborado pelo autor (2024)

Comportamental – Strategy

Esse padrão permite que você crie uma família de algoritmos, encapsule cada um deles e os torne intercambiáveis, assim o algoritmo pode variar independente do cliente que o utiliza.

Vamos ao exemplo. Primeiro definimos a interface da estratégia de cálculo de média (código 23).

```
public interface NotaStrategy {  
    double calculateAverage(double[] grades);  
}
```

Código 23 — Interface da estratégia de cálculo
Fonte: elaborado pelo autor (2024)

Em seguida implementamos as classes concretas de cálculo de média (códigos 24, 25 e 26).

```
public class MediaAritmeticaStrategy implements NotaStrategy {  
    @Override  
    public double calculateAverage(double[] grades) {  
        double sum = 0.0;  
        for (double grade : grades) {  
            sum += grade;  
        }  
        return sum / grades.length;  
    }  
}
```

Código 24 — Classes concretas de cálculo (1)
Fonte: elaborado pelo autor (2024)

```
public class MediaHarmonicaStrategy implements NotaStrategy {  
    @Override  
    public double calculateAverage(double[] grades) {  
        double sum = 0.0;  
        for (double grade : grades) {  
            sum += 1.0 / grade;  
        }  
        return grades.length / sum;  
    }  
}
```

Código 25 — Classes concretas de cálculo (2)
Fonte: elaborado pelo autor (2024)

```
public class MediaPonderadaStrategy implements NotaStrategy {
```

```
private double[] weights;

public MediaPonderadaStrategy(double[] weights) {
    this.weights = weights;
}

@Override
public double calculateAverage(double[] grades) {
    double sum = 0.0;
    double weightSum = 0.0;
    for (int i = 0; i < grades.length; i++) {
        sum += grades[i] * weights[i];
        weightSum += weights[i];
    }
    return sum / weightSum;
}
}
```

Código 26 — Classes concretas de cálculo (3)
Fonte: elaborado pelo autor (2024)

Agora criamos a classe de contexto para uma estratégia (Código 27).

```
public class NotaContext {
    private NotaStrategy strategy;

    public void setStrategy(NotaStrategy strategy) {
        this.strategy = strategy;
    }

    public double executeStrategy(double[] grades) {
        return strategy.calculateAverage(grades);
    }
}
```

Código 27 — Classe de contexto
Fonte: elaborado pelo autor (2024)

Por fim, temos o uso do strategy (código 28).

```
public class Main {  
    public static void main(String[] args) {  
        // Criar o contexto  
        NotaContext context = new NotaContext();  
  
        // Criar diferentes conjuntos de notas  
        double[] grades = {85, 90, 78, 92, 88};  
  
        // Usar a estratégia de média aritmética  
        context.setStrategy(new MediaAritmeticaStrategy());  
        System.out.println("Média aritmética: " +  
            context.executeStrategy(grades));  
  
        // Usar a estratégia de média ponderada  
        double[] weights = {0.1, 0.2, 0.3, 0.2, 0.2};  
        context.setStrategy(new MediaPonderadaStrategy(weights));  
        System.out.println("Média ponderada: " +  
            context.executeStrategy(grades));  
  
        // Usar a estratégia de média harmônica  
        context.setStrategy(new MediaHarmonicaStrategy());  
        System.out.println("Média harmônica: " +  
            context.executeStrategy(grades));  
    }  
}
```

Código 28 — Uso do strategy
Fonte: elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula aprendemos para que servem os padrões de projeto e como eles devem ser aplicados. Vimos também exemplos simples de alguns deles, como base para que o aluno e aluna possam aplicar em seus próprios projetos.



REFERÊNCIAS

CARDOSO, C. **Orientação a Objetos na Prática**. São Paulo: Ciência Moderna, 2020.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books, 2018.

SHVETS, A. **Mergulho nos Padrões de Projeto**. [S.l.: s.n.], 2021.

EMASP

PALAVRAS-CHAVE

Padrões de Projeto. Design patterns. POO.

EMENDAS



POSTECH